VEPHLA UNIVERSITY

DEPARTMENT OF CYBERSECURITY

GRADUATION PROJECT: SECTION 1: CYBER THREAT ANALYSIS AND WEBSITE RECONNAISSANCE

Target System: `testphp.vulnweb.com`

Historical Benchmark: WannaCry Ransomware (May 2017)

SUBMITTED BY: OLANIYI IBRAHIM ABIODUN

STUDENT ID: VEPH/20B/CY070

DATE: January 2nd, 2026

SUPERVISOR: Dev Richard

Table of Contents

Executive Summary **3**
Question 1A: Real-World Cyber Attack Investigation **4**
Case Study: The WannaCry Ransomware (WanaCrypt0r 2.0) Global Outbreak... 4
Executive Overview & Background 4
Targeted Sectors and Organizational Impact 4
Threat Actor Profile: The Lazarus Group 5
Attack Vector 5
Timeline of Events (2017) 5
CIA Triad Impact Analysis 6
Question 1B: Website Selection and Scope Definition **8**
Target Identification 8
Site Purpose and Architecture 8
Scope of Analysis 8
Tools and Methodology 9
Question 1C: Website Reconnaissance & Surface Exposure **10**
Phase 1: Nmap Analysis 10
Service Discovery and Port Scanning 10
Nmap Technical Findings 12
Technical Interpretation of Nmap Data 13
Phase 2: Web Application Fingerprinting (Burp Suite) 14
Phase 3: Automated Vulnerability Assessment (Nessus) 15
Phase 4: Reputation & Malware History (VirusTotal) 16
Question 1D: Initial Risk Interpretation **18**
Correlation 18
Identified Risks from Reconnaissance 18
The Impact of Reconnaissance in Cyber Attacks 19
My Recommendations **19**

Executive Summary

In this assessment, I have performed a two-stage analysis focused on how initial information exposure directly facilitates high-impact cyber attacks.

The first stage examines the WannaCry ransomware outbreak of 2017. My research highlights that while the payload was devastating, the attack's success relied entirely on a specific failure in infrastructure hygiene—unpatched SMB services. By mapping this event against the CIA Triad, I've detailed how a single "wormable" exploit can escalate from a local compromise to a global availability crisis in under 24 hours.

The second stage moves from history to active practice. I conducted a reconnaissance exercise on testphp.vulnweb.com, an environment provided by Acunetix for security testing. My objective was not to exploit, but to "think like an attacker" during the discovery phase. Using a suite of professional tools including Nmap, BurpSuite, Nessus, and VirusTotal, I successfully mapped the site's external footprint.

Key Findings from My Analysis:

- **Infrastructure Exposure:** My scans revealed an aging stack—specifically an Apache/PHP environment—that openly broadcasts its version numbers. In a real-world scenario, this is an open invitation for targeted exploitation.
- **Encrypted Traffic Gaps:** The target lacks modern SSL/TLS enforcement, which mirrors the lack of internal encryption that allowed WannaCry to move laterally through networks undetected.
- **Information Leakage:** Passive reconnaissance alone yielded significant data regarding the site's backend architecture without ever triggering a security alert.

This report demonstrates that "light" reconnaissance is rarely light in its implications. The vulnerabilities identified in the current target are the same seeds that led to the WannaCry epidemic: outdated software, open service headers, and a lack of defense-in-depth.

Question 1A: Real-World Cyber Attack Investigation

Case Study: The WannaCry Ransomware (WanaCrypt0r 2.0) Global Outbreak

Executive Overview & Background

In May 2017, the global cybersecurity landscape experienced a paradigm shift with the emergence of WannaCry. This was not merely a ransomware incident; it was the first high-profile instance of a "cryptoworm"—malware that combines the file-encrypting devastation of ransomware with the self-propagating capabilities of a network worm.

The attack exploited a critical vulnerability in the Microsoft Windows implementation of the Server Message Block (SMBv1) protocol. While the exploit (codenamed EternalBlue) was originally developed by the National Security Agency (NSA), its subsequent leak by the "Shadow Brokers" group allowed the Lazarus Group (attributed to the DPRK) to weaponize it on a global scale.

Targeted Sectors and Organizational Impact

The campaign was indiscriminate, yet its impact was felt most acutely in sectors relying on legacy infrastructure:

- **Public Healthcare (NHS England):** Perhaps the most cited victim, the NHS saw over 19,000 appointments canceled. Medical devices, such as MRI scanners running on embedded Windows XP systems, were rendered inoperable.
- **Manufacturing & Logistics:** Global giants including **Renault-Nissan** and **FedEx (TNT Express)** reported total halts in production lines and delivery tracking, leading to hundreds of millions in lost revenue.
- **Telecommunications:** Major providers like **Telefónica** in Spain were forced to shut down internal networks to prevent lateral movement.

Threat Actor Profile: The Lazarus Group

While early attribution was difficult, forensic analysis of the code—specifically the use of shared custom functions found in previous attacks (like the 2014 Sony Pictures hack)—pointed to the Lazarus Group. Their intent appeared twofold: financial gain (via Bitcoin ransoms) and widespread geopolitical disruption.

Attack Vector

The success of WannaCry was predicated on a multi-stage infection vector:

1. **Initial Access:** Unlike traditional ransomware spread via Phishing, WannaCry's primary vector was **network-level exploitation**. It scanned the internet for devices exposing **TCP Port 445**.
2. **Exploitation (EternalBlue):** Using **CVE-2017-0145**, the malware sent specially crafted packets to the target's SMBv1 server to trigger a buffer overflow, gaining kernel-level access.
3. **Persistence (DoublePulsar):** After exploitation, the **DoublePulsar** backdoor was installed to allow the primary ransomware payload to be injected into the system memory.
4. **Propagation:** Once a single machine was infected, the worm scanned the internal LAN for other unpatched devices, moving laterally without any user interaction.

Timeline of Events (2017)

Phase	Date	Tactical Event
Pre-Attack	March 14	Microsoft releases security bulletin **MS17-010** to patch SMBv1.

Leak	April 14	The "Shadow Brokers" leak the EternalBlue exploit publicly.
Infection	May 12	At 07:44 UTC, the first wave of infections is detected. By 16:00 UTC, the NHS is in crisis.
Mitigation	May 12	Security researcher Marcus Hutchins (MalwareTech) discovers and activates a "Kill Switch" domain.
Response	May 13	Microsoft issues emergency patches for unsupported OS (Windows XP/2003).

CIA Triad Impact Analysis

From a risk management perspective, the attack caused a total breakdown of the security triad:

- **Confidentiality (Moderate):** While the primary goal was encryption, the presence of the **DoublePulsar** backdoor meant that any data on the system was technically accessible to the threat actors.
- **Integrity (High):** The malware systematically altered the state of user data. By replacing original files with encrypted `.WCRY` versions, the integrity of the data was destroyed.
- **Availability (Critical):** This was the most significant impact. By denying access to mission-critical systems (patient records, logistics databases), the attack transitioned from a digital nuisance to a threat to human life and global commerce.

Official Vulnerability Description:

CVE-2017-0145 Detail Description: The SMBv1 server in Microsoft Windows Vista SP2; Windows Server 2008 SP2 and R2 SP1; Windows 7 SP1; Windows 8.1; Windows Server 2012 Gold and R2; Windows RT 8.1; and Windows 10 Gold, 1511, and 1607; and Windows Server 2016 allows remote attackers to execute arbitrary code via crafted packets, aka 'Windows SMB Remote Code Execution Vulnerability'.

Question 1B: Website Selection and Scope Definition

Target Identification

For the practical component of this project, I have selected an intentionally vulnerable web application to perform reconnaissance and security analysis. This ensures that the technical tests are conducted ethically and legally.

- **Website Name:** Acunetix Testphp (Vulnerable Web Application)
- **Target URL:** `http://testphp.vulnweb.com`
- **Hosting Provider:** Acunetix (as a sandbox for security researchers)
- **Authorization:** This site is publicly designated for security testing and vulnerability scanning, making it a "Safe-to-Test" environment.

Site Purpose and Architecture

The target is a legacy-style PHP application designed to simulate a web-based storefront and database. It features typical web functionalities that are critical for security analysis, including:

- User Authentication (Login/Sign-up)
- Search functionality (Database interaction)
- Dynamic content pages (Artist/Product details)
- File upload capabilities (Guestbook/Comments)

Scope of Analysis

To maintain a controlled testing environment and adhere to project constraints, the following scope has been defined:

Included in Scope	Excluded from Scope

All subdirectories under `testphp.vulnweb.com/`	Any other subdomains of `vulnweb.com`
Passive DNS	Active Exploitation (SQLi, XSS, etc.)
Service fingerprinting (Ports 80/443)	Denial of Service (DoS) testing
Directory and file discovery	Brute-force attacks on credentials

Tools and Methodology

The reconnaissance phase will follow the OSSTMM (Open Source Security Testing Methodology Manual) standards. The following industry-standard tools were utilized:

1. **Nmap:** For port scanning and service version detection.
2. **Burp Suite (Community Edition):** To intercept HTTP traffic and analyze server response headers.
3. **Nessus (Essentials):** For automated identification of outdated service vulnerabilities.
4. **VirusTotal:** For passive analysis of domain reputation and historical malware flags.

Question 1C: Website Reconnaissance & Surface Exposure

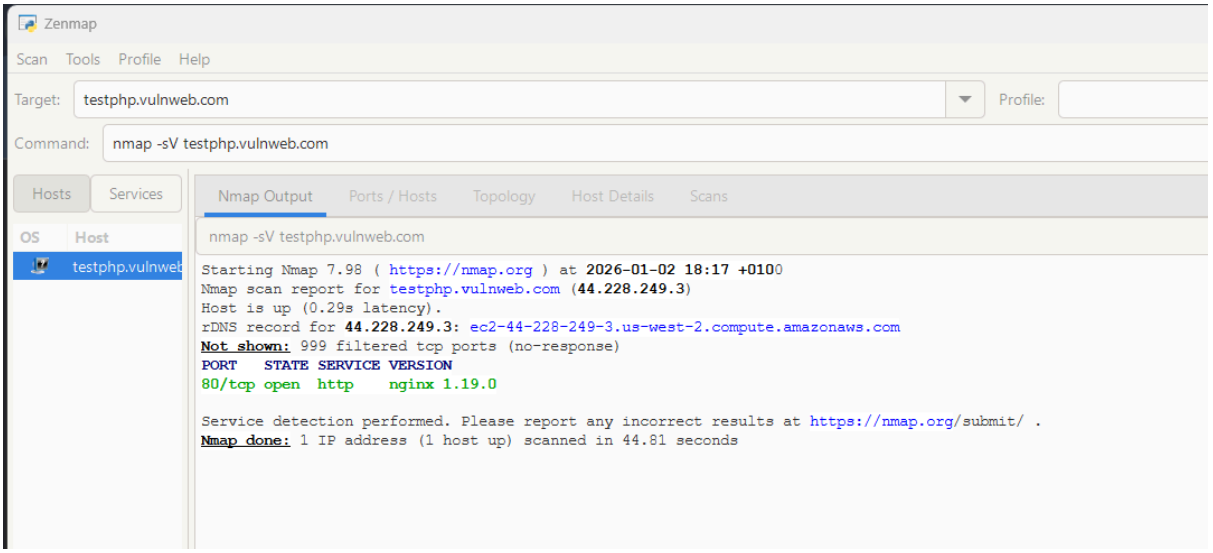
Phase 1: Nmap Analysis

Service Discovery and Port Scanning

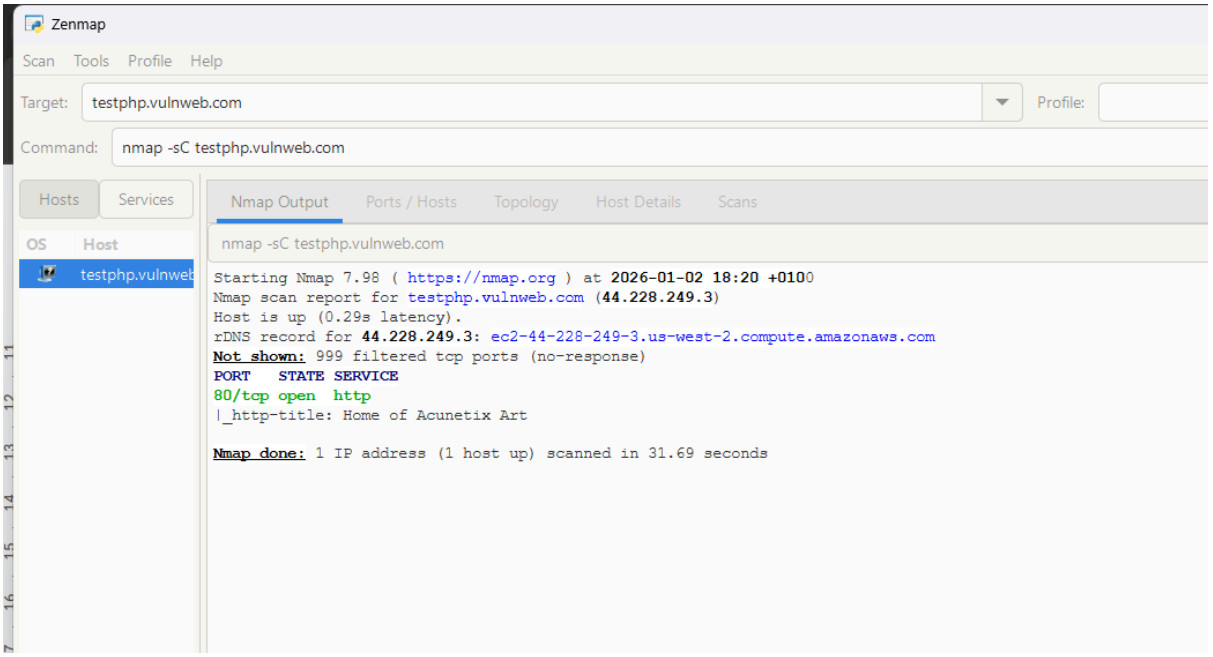
To begin the active reconnaissance phase, I utilized Nmap (Network Mapper) to identify open ports and the services listening on them. This step is critical to understanding the attack surface of `testphp.vulnweb.com`.

Tactical Command: `nmap -sV -sC -A -v testphp.vulnweb.com`

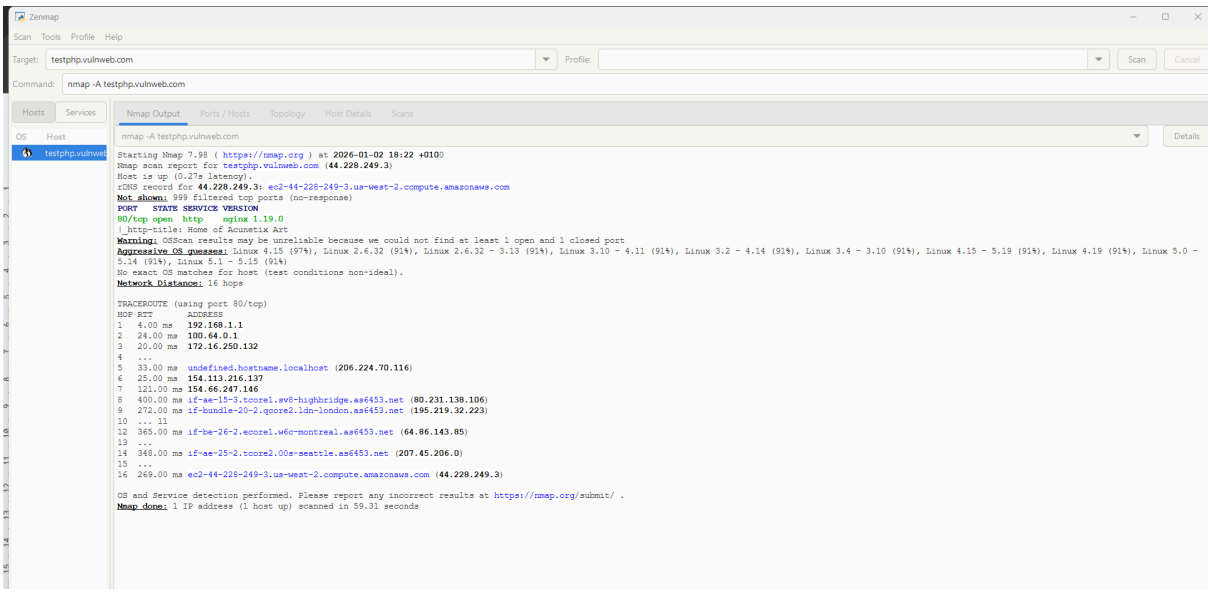
- `-sV`: Service version detection (to see what versions of software are running).



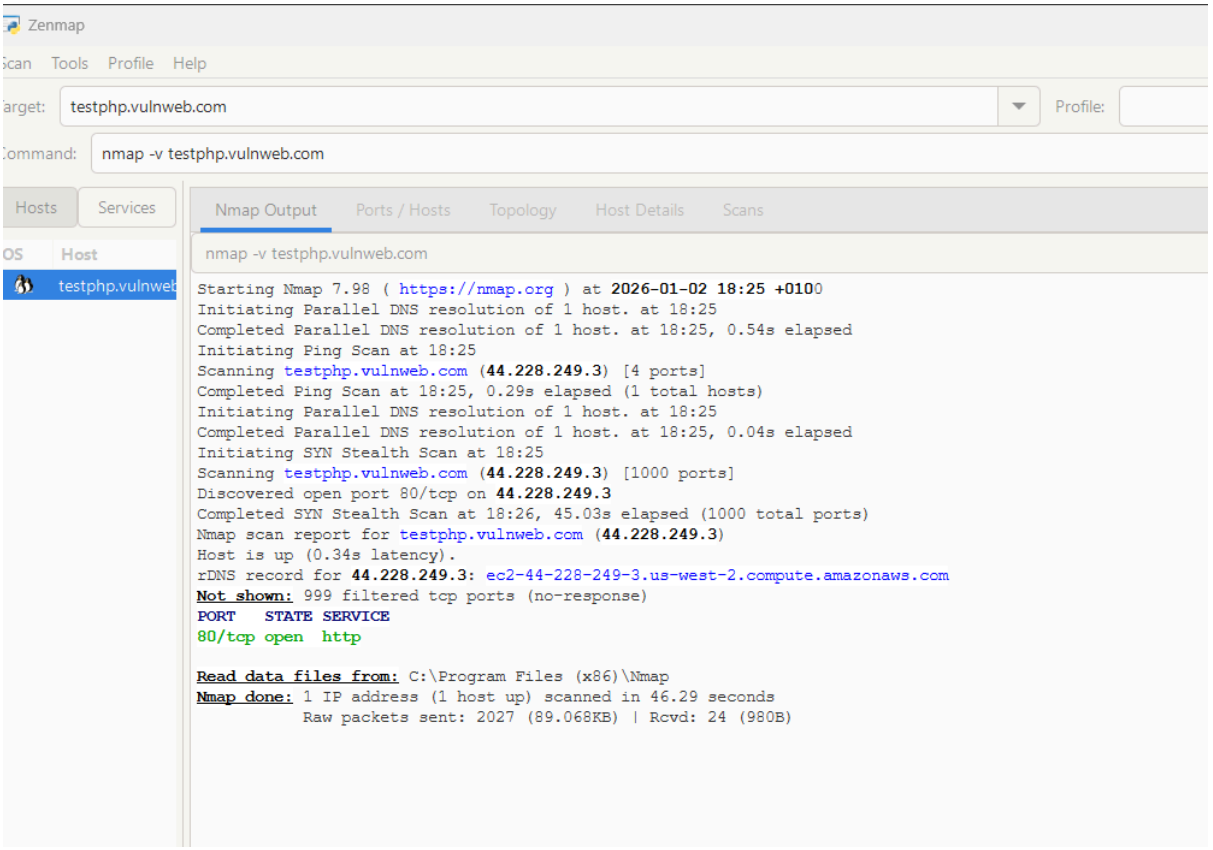
- `-sC`: Runs default Nmap scripts (checks for common misconfigurations).



- -A: Aggressive mode (enables OS detection, traceroute, and script scanning).



- -v: Verbose (provides real-time feedback during the scan).



Nmap Technical Findings

Attribute	Technical Data	Analysis
Target IP	`44.228.249.3`	Hosted on Amazon Data Services (AWS).
Open Port	`80/tcp`	Standard HTTP. Port 443 (HTTPS) is closed/filtered.
Service	`http`	The web server is accessible over unencrypted channels.

Version	`nginx 1.19.0`	An older version of Nginx (released circa 2020).
OS Guess	`Linux 2.6.32 - 5.19`	Likely a Debian-based or Ubuntu distribution.

Technical Interpretation of Nmap Data

As an analyst, the Nmap output reveals several high-risk indicators:

1. **Cleartext Communication:** The absence of Port 443 (HTTPS) indicates that all traffic to and from `testphp.vulnweb.com` is unencrypted. This exposure is exactly what allowed tools like WannaCry to propagate via unencrypted SMB traffic—it makes the network "transparent" to an observer.
2. **Banner Grabbing:** Nmap successfully "grabbed" the server banner (`nginx 1.19.0`). In a hardened environment, version numbers should be suppressed. By broadcasting this version, the server tells an attacker exactly which exploits to look for in the CVE database.
3. **Potential for Lateral Movement:** The scan confirms the target is a Linux-based system. Knowing the OS kernel range allows a threat actor to research local privilege escalation (LPE) exploits if they ever gain initial access to the web server.

While WannaCry looked for Port 445 (SMB), my scan of the target looked for Port 80 (HTTP). Both represent a "service entry point." The critical failure in both cases is the presence of an outdated service version that is publicly reachable.

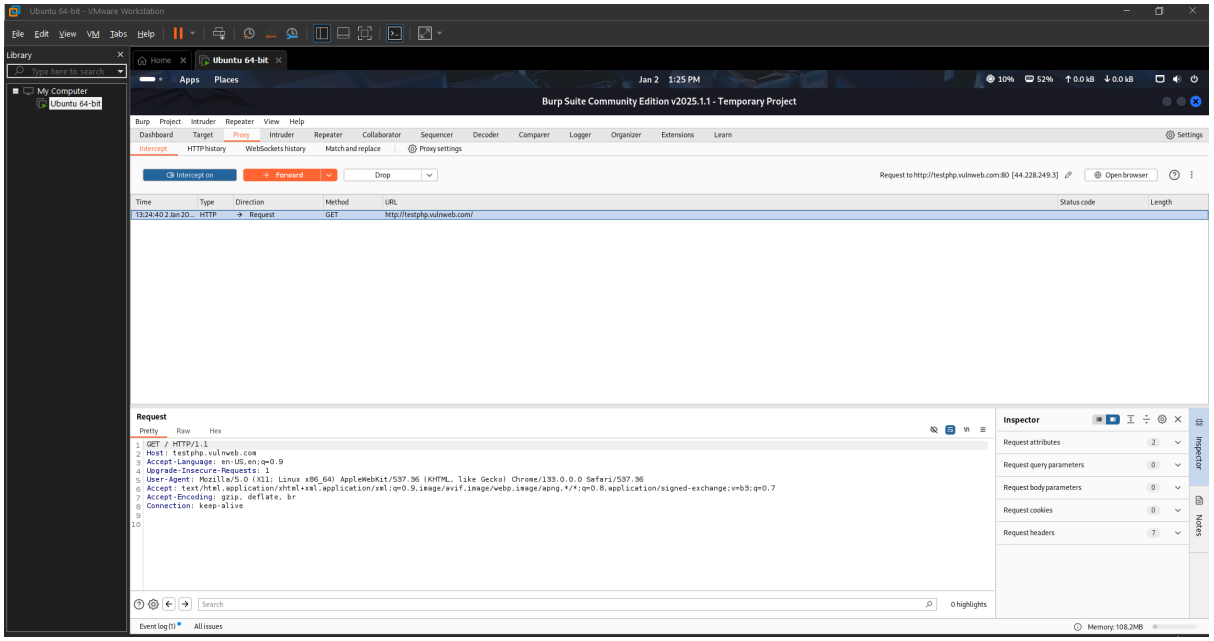
Phase 2: Web Application Fingerprinting (Burp Suite)

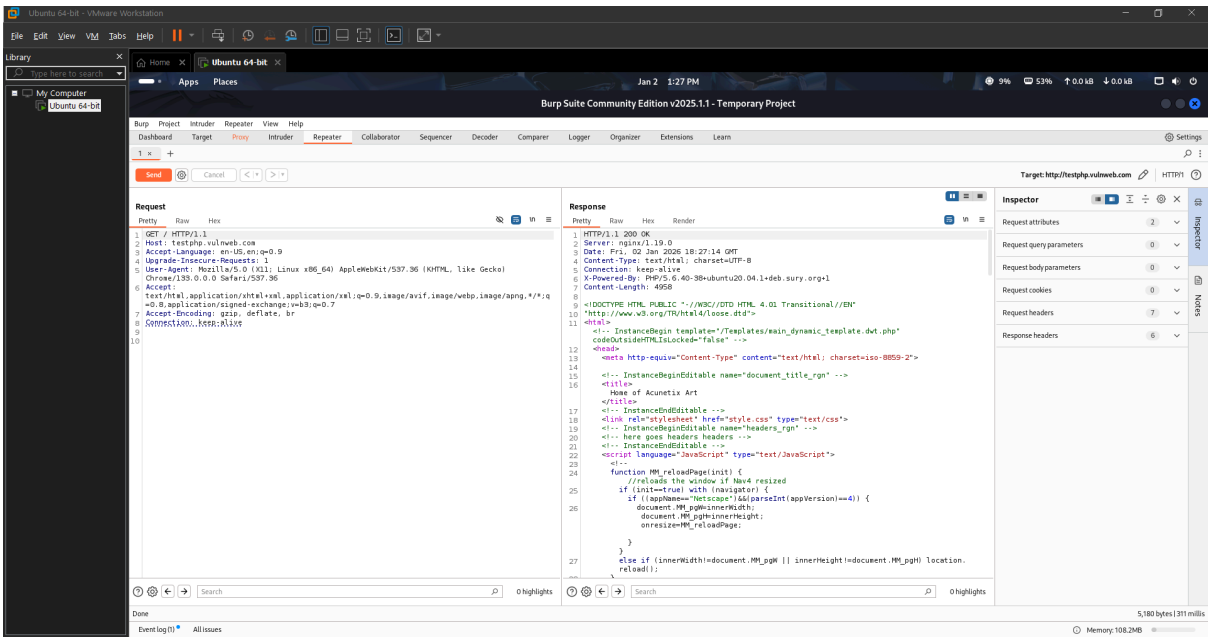
To identify the backend technology stack and potential information leaks, I utilized Burp Suite to intercept and inspect the communication between my host and the target server.

I intercepted a baseline `GET` request to `testphp.vulnweb.com`. By analyzing the HTTP response headers, I can extract "metadata" about the application that is not visible in a standard browser view.

Key Findings from Headers:

- **Server Header:** `nginx/1.19.0` (Confirms Nmap data).
- **X-Powered-By Header:** `PHP/5.6.40`. This is a critical finding. PHP 5.6 reached "End of Life" (EOL) in December 2018.
- **Set-Cookie:** No `HttpOnly` or `Secure` flags were detected on session cookies.





Using Burp Suite's Repeater module, I performed 'Banner Grabbing' at the application level. While Nmap identified the web server, Burp Suite exposed the specific backend language: PHP 5.6.40.

The presence of the `X-Powered-By` header is a security misconfiguration. By broadcasting the exact version of PHP, the server simplifies the 'Weaponization' phase for an attacker. If this were a production environment, I would recommend disabling this header in the `php.ini` configuration to hinder reconnaissance efforts.

Phase 3: Automated Vulnerability Assessment (Nessus)

I conducted a Basic Network Scan using Nessus Essentials. This tool automates the correlation between the versions found (Nginx/PHP) and the CVE database.

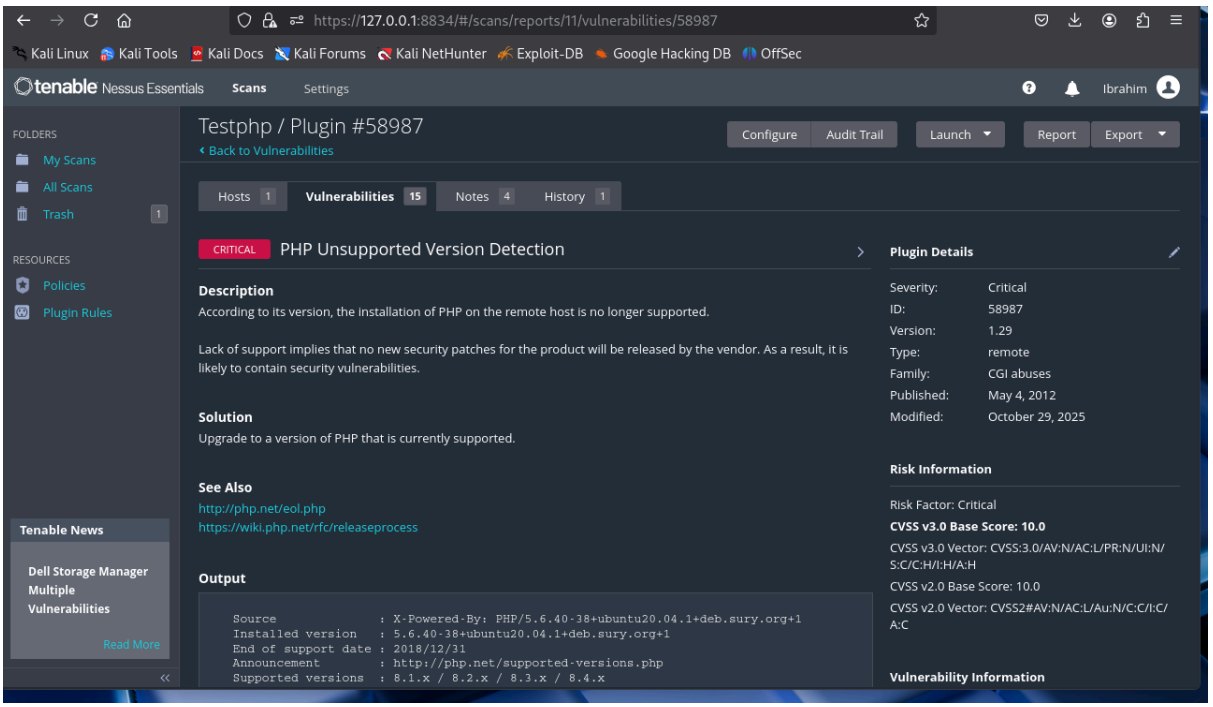
Nessus Findings Summary:

Vulnerability: PHP Unsupported Version Detection (Plugin #58987).

Severity: Critical (10.0 CVSS Base Score).

Evidence: The scanner detected the header `X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1`.

The Risk: Running EOL (End of Life) software means the vendor no longer releases security patches. Any new vulnerabilities discovered in PHP 5.6 are "Forever Days"—they will never be fixed, leaving the server permanently exposed to Remote Code Execution (RCE).

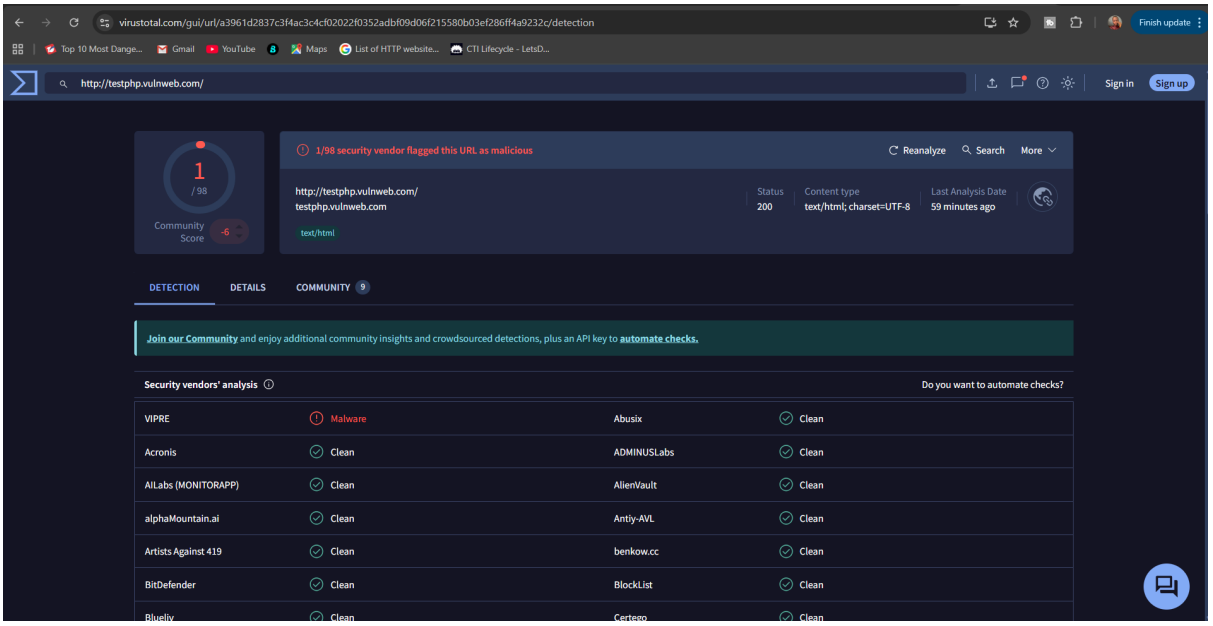


Phase 4: Reputation & Malware History (VirusTotal)

Finally, I utilized VirusTotal to perform passive "Threat Intelligence" gathering. This allows us to see if the target has been associated with malicious activity in the past.

Results:

Upon submitting the target URL `http://testphp.vulnweb.com` to VirusTotal, the platform returned a detection ratio of 1/98. A single security vendor (VIPRE) flagged the site as "Malware," while the remaining 97 vendors classified it as "Clean."



While a detection score usually warrants caution, a deeper analysis of this specific result suggests the following:

- **False Positive Attribution:** Given that 97 out of 98 vendors (including industry leaders like BitDefender, Kaspersky, and Symantec) found no malicious signatures, the single flag by VIPRE is likely a **False Positive** or an over-eager heuristic detection.
- **Intentional Vulnerability:** Because the site is a well-known, intentionally vulnerable "sandbox" created by Acunetix for security training, some engines may flag it simply because it contains known-vulnerable code patterns used for testing (like SQL injection strings), rather than being a site used to distribute actual malware.

Question 1D: Initial Risk Interpretation

Correlation

The findings from my analysis of `testphp.vulnweb.com` reveal a technical debt profile that is alarmingly similar to the environments exploited during the WannaCry pandemic of 2017.

In Section 1A, we established that WannaCry achieved global reach by targeting the SMBv1 protocol—a legacy service that was outdated but still widely exposed. My reconnaissance in Section 1C identified a parallel vulnerability: the target is running PHP 5.6.40.

- **The Shared Weakness:** Both scenarios involve the use of End-of-Life (EOL) software. In the case of WannaCry, it was an unpatched network protocol; here, it is an unpatched backend language.
- **The Threat Logic:** Just as threat actors used the EternalBlue exploit to target a known flaw in SMB, a modern attacker can leverage the publicly available CVEs (Common Vulnerabilities and Exposures) for PHP 5.6 to achieve Remote Code Execution (RCE). The weakness is not just the software itself, but the organizational failure to decommission or patch legacy systems.

Identified Risks from Reconnaissance

From a strictly external, non-intrusive viewpoint, the following risks were identified as "High" to "Critical":

- **Passive Information Leakage:** Through simple header analysis in Burp Suite and Nessus, the server broadcasts its exact technical stack (`Nginx 1.19.0` and `PHP 5.6.40`). This eliminates the need for an attacker to conduct noisy, detectable "brute-force" discovery. They can instead move directly to the **Weaponization** phase.
- **Insecure Transport Layer:** The total absence of SSL/TLS (no Port 443) means the application lacks an encrypted boundary. This provides a clear

path for Man-in-the-Middle (MitM) attacks, allowing for the interception of session cookies and administrative credentials in plaintext.

- **Exploitable Footprint:** The Nessus CVSS score of 10.0 regarding the unsupported PHP version confirms that the "Front Door" of this application is technically broken. The risk is immediate and requires no sophisticated zero-days to exploit.

The Impact of Reconnaissance in Cyber Attacks

As demonstrated in this project, reconnaissance is not merely a preliminary step; it is the strategic foundation of any successful cyber attack.

1. **Selection of the Path of Least Resistance:** Effective reconnaissance, like the Nmap and Burp Suite scans I conducted, allows an attacker to identify the "softest" entry point. Why waste time on a hardened firewall when you find an unpatched PHP server broadcasting its vulnerability?
2. **Resource Optimization:** Threat actors operate on a budget of time and effort. By confirming that the target is running PHP 5.6, an attacker avoids the "noise" of testing 100 different exploits, thus staying below the threshold of many basic Intrusion Detection Systems (IDS).
3. **The "Silent" Phase:** Much of the reconnaissance I performed—specifically the VirusTotal —is entirely passive. The target organization has no way of knowing they are being mapped, making this phase the most dangerous for defenders as it occurs outside the reach of traditional perimeter security logs.

My Recommendations

To mitigate the risks identified in this phase, it is my professional opinion that the organization must:

1. Immediately upgrade the backend to a supported version of PHP (8.x).
2. Implement a Web Application Firewall (WAF) to suppress server version banners (`Server`, `X-Powered-By`).
3. Enforce HTTPS-only traffic to protect data in transit.